TubeBench: Protocol-Validated Evaluation of Browser Agents on Long-Form Media Tasks

Venkata Hemanth Athota

June 2026

Abstract

AI co-work systems increasingly operate through browsers, media players, transcripts, captions, and timeline controls, but existing evaluations often reduce performance to static question answering, coding tasks, or short GUI interactions. TubeBench is a harness-agnostic benchmark for evaluating browser-use agents and model-tool environments on long-form content workflows across media players and content platforms. The benchmark measures whether an agent can navigate temporal media, manipulate player state, use transcript and metadata evidence, ground final answers in observed browser state, recover from interaction errors, and produce replayable traces for scoring. This paper reports the completed benchmark design, deterministic diagnostic controls, and static protocol-validation evidence currently present in the repositories. The evidence shows that trace-level scoring can separate top-level task completion from grounding gaps, side effects, trace-transfer failures, runtime blockers, and overconfident finalization. The results do not establish broad benchmark-level performance for any one harness; instead, they characterize a reproducible measurement substrate and the failure modes that larger controlled evaluations must preserve rather than hide.

1 Introduction

Long-form media tasks are a common form of delegated browser work. A user may ask an AI co-work system to find a relevant moment in a lecture, pause the only active player, compare evidence across videos, use a transcript to locate a claim, or answer a question while preserving unrelated browser state. These tasks are not simply video question answering: the agent must operate inside a live browser environment, choose among controls and evidence channels, track temporal state, and verify that its visible actions produced the intended result.

TubeBench addresses this setting by treating browser-media work as an auditable systems workflow. The benchmark records task specifications, browser-visible observations, media-control actions, trace capture, evidence grounding, side effects, verification events, scoring outputs, and failure categories. It is harness-agnostic: Claude-based co-work tooling, Codex-like environments, browser agents, GUI agents, and other agentic model-tool systems can all be evaluated when they expose the required traces or manual handoff artifacts.

This paper makes four contributions:

1. a benchmark framing for long-form browser-media tasks that require temporal navigation, media-player control, evidence extraction, and final-state verification;
2. a task and trace protocol that separates task completion, partial completion, grounding, safety, recovery, and infrastructure blockers;
3. diagnostic-control and static protocol-validation results that demonstrate the publication and scoring pipeline without presenting them as model capability estimates; and
4. a failure taxonomy for interaction, grounding, verification, trace-transfer, and runtime-controller failures in browser-use agent evaluation.

2 Related Work

TubeBench builds on web and GUI-agent evaluation while targeting a narrower interaction regime. WebArena evaluates long-horizon web tasks in realistic websites [6]; VisualWebArena emphasizes visually grounded web tasks [3]; Mind2Web studies generalist agents on real websites [1]; and OSWorld evaluates agents in open-ended computer

environments [4]. SWE-bench is not a browser-media benchmark, but it motivates executable, evidence-backed task evaluation [2]. Recent living-screen GUI work further highlights the volatility of media-centered interfaces [5].

The gap addressed by TubeBench is the intersection of browser action, temporal media understanding, and trace-level scoring. Static document QA can test reading, but not media-control execution. Web QA can test navigation, but often omits player state and temporal observation. Coding benchmarks test repository-grounded problem solving, but not browser-mediated evidence extraction. General GUI tasks may test clicking and form filling without requiring long-horizon media grounding. TubeBench isolates tasks where the agent must interact with long-form content over time.

3 Benchmark Design

3.1 Task Scope

TubeBench tasks cover long-form media and browser-media workflows rather than a single platform. A task may involve a hosted benchmark fixture, a local media-player surface, or a public content platform when policy and reproducibility constraints permit. The core task families are:

- locating relevant moments in long-form content;
- controlling media players, including play, pause, seek, mute, and restore operations;
- using transcripts, captions, chapters, metadata, and visual context as evidence;
- grounding a final answer in observed browser and media state;
- preserving unrelated tabs, players, and account-visible state; and
- recovering from navigation or interaction errors without erasing the failure record.

3.2 Task Protocol

Each task specifies a browser-media environment, allowed and forbidden actions, expected evidence, success predicates, verification requirements, and scoring criteria. A trace records the agent's observations and actions, browser/tool calls, state snapshots, watched intervals, evidence references, side effects, errors, and final answer. The protocol supports binary predicates where the state is exact, but it also records partial and ordinal outcomes when exact scoring would overstate the evidence.

The static TCE-002 artifact is used in this paper only as an internal example of the protocol: it asks the agent to pause the one playing player while preserving other players. The reusable contribution is not that one page or endpoint; it is the schema pattern for browser-visible action, trace handoff, state verification, and retained-slot accounting.

3.3 Evaluation Pipeline

Figure 1 shows the benchmark pipeline. The harness operates in the browser/media environment, records a trajectory, and hands it to a scorer. The scorer produces task-level and trace-level results that feed error analysis. Publication receives aggregate, reviewed artifacts only.

4 Metrics and Scoring

TubeBench reports trace-level metrics rather than a single final-answer score. Table 1 summarizes the core dimensions used in the current paper.

For tasks with exact state predicates, the evaluator records exact success:

$$S_{\text{exact}} = \mathbf{1}[\text{all required final predicates hold}].$$

The stricter disturbance-free score requires success, required evidence, and no side-effect incident:

$$S_{\text{dfs}} = \mathbf{1}\left[S_{\text{exact}} = 1 \wedge E_{\text{required}} = 1 \wedge I_{\text{side effects}} = 0\right].$$

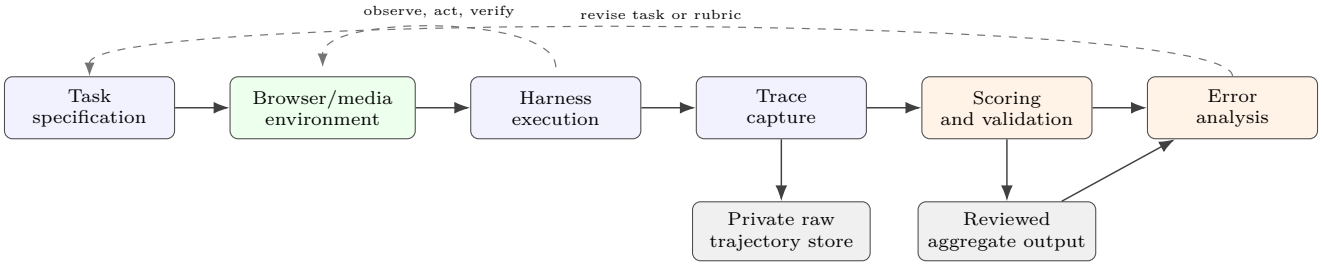

Figure 1: TubeBench evaluation pipeline. The benchmark separates task specification, browser/media execution, trace capture, scoring, and error analysis so that partial completion, grounding failures, side effects, and runtime blockers remain visible.

Table 1: Core scoring dimensions for long-form browser-media tasks.

Dimension	Values	Interpretation
Task completion	complete / partial / failed	Whether required final-state predicates or answer criteria were satisfied.
Evidence grounding	grounded / partial / ungrounded	Whether the final claim is supported by observed transcript, metadata, visual, or state evidence.
Navigation success	complete / partial / failed	Whether the agent reached the relevant media context without losing the task surface.
Media-control success	complete / partial / failed	Whether player actions changed the intended media state and preserved unrelated state.
Transcript-use accuracy	high / medium / low / not applicable	Whether transcript or caption evidence was used correctly when needed.
Recovery from error	recovered / brittle / failed / not exercised	Whether the trajectory handled incorrect actions or runtime friction without hiding them.
Final answer correctness	correct / partial / incorrect / not applicable	Whether the user-facing answer matches the evidence and task.
Trajectory validity	valid / invalid / blocked	Whether the trace is ingestible, policy-compliant, and scoreable.

These formulas are useful for deterministic fixtures, but TubeBench treats them as part of a larger trace record. A run can be task-complete but weakly grounded, partially recovered, invalid as a trace, or blocked by browser runtime failure.

5 Experiments

5.1 Diagnostic Controls

The deterministic diagnostic aggregate contains four control conditions with 72 attempts each. These are not claims about any deployed model or co-work product. They test whether the scorer, aggregation code, claims ledger, and paper pipeline preserve denominators and labels. Table 2 reports the current aggregate.

The reference conditions are intentionally controlled. They verify pipeline behavior: denominators are retained, failed controls are not upgraded into successes, and task success remains separate from disturbance-free behavior. They do not establish that any interactive browser agent achieves these rates.

5.2 Static Protocol Validation

Two retained-slot static protocol-validation campaigns are recorded as aggregate evidence. The earlier repetition produced four valid task-started traces and one runtime blocker. The readiness-gated repetition passed five readiness gates but then recorded five pre-interaction runtime blockers before task interaction.

Table 2: Diagnostic-control aggregate. Curated reference trajectories are comparison points for expected task completion under controlled assumptions, not claims of exhaustive reasoning or complete ground truth.

Condition	Attempts	Task completion	Disturbance-free score	Role
No-op control	72	4.2%	4.2%	Negative diagnostic: failures remain failures.
Planner reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Skill reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.
Hybrid reference	72	100.0%	100.0%	Oracle-assisted reference trajectory.

Table 3: Static browser-media protocol-validation outcomes. These results measure trace handoff and browser-controller reliability, not broad task performance.

Campaign	Slots	Readiness passed	Interactions	Valid traces	Runtime blockers	Interpretation
Earlier	5	not separately gated	4	4	1	Four valid task-started traces; strict stability false.
Readiness-gated	5	5	0	0	5	Readiness passed; tab lost before action.

For a retained campaign of N slots, TubeBench records:

$$r_{\text{ready}} = \frac{n_{\text{ready}}}{N}, \quad r_{\text{valid}} = \frac{n_{\text{valid traces}}}{N}, \quad r_{\text{blocked}} = \frac{n_{\text{runtime blocked}}}{N}.$$

For the readiness-gated campaign, $r_{\text{ready}} = 1.0$, $r_{\text{valid}} = 0.0$, and $r_{\text{blocked}} = 1.0$. Strict handoff stability is therefore false. The important finding is the layer at which failure occurs: the task did not fail because the agent misunderstood media state; the protocol exposed an infrastructure failure before task interaction.

6 Results and Error Analysis

The completed evidence supports three observations.

Trace-level scoring is more informative than binary success. The deterministic controls show why final-state predicates and disturbance-free behavior must be reported separately. A benchmark that records only the final answer would miss side effects, restoration gaps, and weak verification.

Protocol validation exposes infrastructure failures. The static campaigns preserved runtime blockers and trace-capture outcomes as first-class records. This matters because replacing failed slots would bias a reliability study toward successful handoffs and erase the actual browser-controller failure mode.

Current evidence is not a broad model-performance result. The available aggregates validate measurement machinery and protocol behavior. They do not show stable, repeated performance for a named model, harness, or co-work product across the full task space.

Table 4 gives the failure categories used to preserve these distinctions.

7 Discussion

TubeBench is designed for evaluations where the path matters. In long-form media workflows, the user-facing risk is often not just a wrong answer. An agent may disturb another player, rely on an unavailable transcript, skip verification, lose the task tab, or report success after a partial recovery. Preserving these events in the trace makes the evaluation more useful for improving browser-use systems.

The same design choice limits what this paper claims. Diagnostic controls are necessary because they show that the pipeline can produce, aggregate, and publish results; they are not agent-performance evidence. Static protocol validation is useful because it tests handoff and browser-runtime reliability; it is not a substitute for a diverse task suite. The current results therefore support readiness and methodology claims rather than a leaderboard claim.

Table 4: Failure modes for browser-media agent evaluation.

Failure class	Observable signal
Task misunderstanding	The agent targets the wrong media item, time span, player, or state predicate.
Navigation failure	The agent cannot reach or retain the relevant browser/media context.
Media-control failure	The agent invokes the wrong control or changes the wrong player state.
Evidence-grounding failure	The final claim is unsupported, partially supported, or contradicted by observed evidence.
Transcript or metadata misuse	Transcript, captions, chapters, or metadata are treated as sufficient when required media evidence is absent or conflicting.
Weak verification	The agent declares completion without checking the required state or evidence.
Side-effect disturbance	The run changes unrelated tabs, players, account-visible state, or protected browser state.
Brittle recovery	The agent partially recovers from an error but leaves state, evidence, or reasoning inconsistent.
Trace-transfer failure	Browser-visible work occurs but the trace cannot be captured, copied, ingested, or scored.
Runtime/controller failure	Browser, tab, controller, or harness lifecycle failure prevents or interrupts task interaction.
Overconfident finalization	The final answer masks partial completion, missing evidence, or infrastructure blockers.

8 Limitations

The current paper uses evidence available in the repositories. The deterministic controls are synthetic or oracle-assisted diagnostics. The static protocol evidence is focused on one internal task identifier and one handoff path. Live public-platform evaluation remains volatile because media pages, transcripts, ads, consent flows, recommendations, and live edges can change over time. The paper excludes raw traces, screenshots, account state, browser history, credentials, cookies, and private captures. Exact scoring can reject semantically plausible alternatives if the task schema is too narrow, while ordinal scoring requires careful review to avoid vague success inflation.

9 Artifact Availability

The released paper artifact is intended to be served as a PDF at the public PDF endpoint. The source LaTeX project, claims ledger, and aggregate-safe paper inputs are maintained in the paper repository; raw browser traces remain outside the paper artifact boundary.

10 Conclusion

TubeBench provides a harness-agnostic benchmark and reporting framework for browser-use agents operating on long-form media tasks. The current evidence demonstrates a trace-level evaluation substrate that can distinguish complete, partial, failed, invalid, and infrastructure-blocked outcomes. This is the necessary basis for future controlled comparisons of AI co-work systems, but it does not yet establish broad performance for any one model or harness. The main result is methodological: long-form browser-media evaluation needs task traces, evidence grounding, side-effect accounting, and retained failure records to be scientifically useful.

References

[1] Xiang Deng et al. Mind2web: Towards a generalist agent for the web, 2023. URL `https://arxiv.org/abs/2306.06070`.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL `https://arxiv.org/abs/2310.06770`.

[3] Jing Yu Koh et al. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. URL `https://arxiv.org/abs/2401.13649`.

[4] Tianbao Xie et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

[5] Jiashu Yao, Heyan Huang, Daiqing Wu, Wangke Chen, Huaxi Ai, Haoyu Wen, Zeming Liu, and Yuhang Guo. Benchmarking living-screen-native gui agents on short-video platforms, 2026. URL https://arxiv.org/abs/2606.04701.

[6] Shuyan Zhou et al. Webarena: A realistic web environment for building autonomous agents, 2023. URL https://arxiv.org/abs/2307.13854.